

petta-memory Deployment Plan — Iter/ProtoCosmo2 Integration

Status: proposal for review — nothing in this document has been applied to the runtime yet. Author: Iter agent (ProtoCosmo2 VM). Date: 2026-09-02.
Audience: Ben + frontier-model reviewer.

1. Purpose and scope

Deploy the petta-memory medium-memory store as the Iter agent's persistent memory layer, in gated phases. This document describes exactly what changes are planned, where they land, how they are gated, and how to roll them back. It covers Phase P2 (runtime hook) as the only change requiring a code edit to `iter.py`; all other items are configuration, convention, or already-complete validation work.

2. Current verified state (ground truth)

- **petta-memory repo:** `projects/petta-memory/repos/petta-memory`, branch `agent/parser-validation @ 7aa6b16`, clean, pushed to GitHub. `origin/main = ce5e206`; remote `agent/protocosmo2-handoff @ 5b842f4` is an ancestor of `7aa6b16`. Test suite: **731 tests + 206 subtests PASS**.
- **query_type fix** (commit `7aa6b16`): union semantics — `atom-head OR ClusterType` value match. Fixes Promotion-cluster lookup. Regression test included.
- **Runtime:** `patham9/iter` clone @ `f4064d9`, running at `projects/omegaclaw/protocosmo2/iter-port/repos/`. Untracked local additions: `tools/read_only_status.py`, `channels/protocosmo2.py`. `memory/` system-prompt budget: 3000 chars; files prefixed `_` excluded. **iter.py edits require an agent restart.**
- **Journal:** `memory/_journal/journal.metta`, currently 63 clusters (39 Episodes / 15 Facts / 9 Promotions). PeTTa-load verified. Last full audit: 0 orphans / 0 duplicates / 0 dangling references. Conventions: `timezone.utc` ISO stamps (local clock is UTC+7); Episode→Fact→Promotion→Belief layering; stale claims superseded, never edited (a Supersedes link + re-grounded fact/belief).
- **Completed phases:** P0 (store API hardening: atomic `append_cluster` with validation), P1 (journal bootstrap), P3 (context assembly re-validated @21:42 vs patched store), P4 (promotion schema validated; live promotion `promo-store-api-core → bel-store-api-core`; GoalChainer handoff contract verified — `goalchainer_handoff_cache`, `BeliefContent (Acceptable action-id)`, `PromotionDomain goalchainer`; E2E `solve_incident(memory_items=...)` fused journal evidence with measurable STV shift `0.049/0.855 → 0.004/0.9986`).
- **Side track (GoalChainer repo):** `origin/main = 006d525` (PR#1 merge landing `79ea049` PeTTaChainer integration). Local `main = ce7c340` (merge, 2-hunk union resolution in `metta_reasoner.py`; external/`PeTTaChainer` submodule @ `36b3fe68` initialized). Suite on merged tree: **142p/6s/4f in 135s** — the 4 failures are exactly the known `test_backward_premise_prefilter*` PeTTa failures (positive control confirmed: disabling the prefilter causes findall KB explosion; prefilter-ON exits 0). Local `main` ahead of origin by 6 commits; **push decision left to Ben.**

3. Proposed changes

3.1 Upstream hygiene (zero runtime risk)

Open PR `agent/parser-validation → main` in `petta-memory` so the `query_type` union-semantics fix and `append_cluster` atomic validation are reviewable/mergeable independently of any runtime work.

3.2 P2 — runtime hook (the only iter.py code change)

Gate: environment variable ITER_PETTA=1, default **OFF**. Path to the petta-memory src/ tree supplied via ITER_PETTA_SRC (PYTHONPATH-style import — **no vendoring**, **no copy** of petta-memory into the iter tree).

Call site: after save_experience(experience) at iter.py:322, guarded:

```
if os.environ.get("ITER_PETTA"):
    petta_journal_step(event_append, [c.function.name for c in message.tool_calls])
```

The hook body (draft exists at memory/_drafts/petta_hook.py, 52 lines) appends **one Episode cluster per agent step** to memory/_journal/journal.metta: - IDs: ep-<utc-timestamp-μs> / ev-<utc-timestamp-μs> (collision-free by construction). - Content: user-input snippet + tool names, sanitized (_clean: strips quotes/backslashes/newlines, caps 160 chars). - Evidence link: EvidenceFor fact-step-log <episode>; the carrier fact fact-step-log is created once if absent. - **Failure isolation:** the entire append is wrapped in try/except; any error prints petta hook skipped: ... and the agent turn continues unaffected. The hook can never raise into the main loop.

3.3 P3 — context injection (transformation, no iter.py edit)

memory/_drafts/petta_context.py is an iter **transformation** module (drop into transformations/): before each model call it appends a system message containing a bounded journal **index view**: - Hard cap **1500 chars**, tail-biased (newest clusters kept; truncation is line-aligned, never mid-line). - No-op when ITER_PETTA_SRC unset, journal missing, or on any exception (returns inputs unchanged; iter.py additionally error-isolates transformations). - Re-validated @21:42 against the patched store with a 55-cluster journal; index tail under cap, newest entries present.

3.4 Companion read-only tool

memory/_drafts/petta_recall.py (drop into tools/): query interface mode=index|about|cluster. Read-only, never writes, returns error strings instead of raising. Known API caveat handled: query_about default limit=20 silently truncates — the tool passes explicit large limits.

3.5 Operational conventions (no code)

- Journal is **append-only**; every append goes through append_cluster (atomic, validated: requires MemoryCluster, SchemaVersion=medium-memory-v1, ClusterType, ClusterOpenedAt, Contains, ClusterSource; globally unique IDs; link arity enforced; reload-safe).
- One journal audit per session end (orphans/dups/dangling refs).
- Corrections use Supersedes + new fact (exercised 3x on 2026-09-02, including one self-correction of a garbled episode).

3.6 Later roadmap items (out of scope for this change set)

1. petta-memory + GOAL-SYSTEM fusion: wire GoalChainer memory_items= path against the journal in a canary thread first (contract already verified — this is integration testing, not schema work).
2. GOLEM-Iter.
3. PMC core cycle.

4. Activation and rollback

Activation checklist: (1) Ben's explicit in-channel go-ahead; (2) pick a window coinciding with an already-planned iter restart (edits require restart); (3) apply the 3-line iter.py guard + paste hook function; (4) drop petta_context.py into transformations/ and petta_recall.py into tools/; (5) set ITER_PETTA=1 and ITER_PETTA_SRC=<petta-memory>/src in the service environment; (6) restart; (7) re-run canaries (drafts re-validated post-query_type fix @21:41: hook live-run +2 clusters, carrier no-dup; canaries PASS). **Rollback:** unset ITER_PETTA (hook off) and remove the two drop-in files. No code revert needed; the journal file is inert data when nothing reads it. Worst case: git checkout iter.py — the diff is one guard + one function.

5. Risks and mitigations

Risk	Mitigation
Hook failure breaks agent turns	try/except around entire append; print-and-continue
Journal growth unbounded (63c now; 600+ later)	1500-char tail-biased context cap; compaction policy TBD (flagged, not blocking)
Silent query truncation	explicit <code>limit</code> everywhere (default is 20)
Concurrent writers corrupting journal	single-writer assumption: <code>iter</code> writes only between turns; <code>append_cluster</code> is atomic
<code>iter.py</code> edit needs restart	schedule with a planned restart
Clock skew (UTC+7 local)	all stamps <code>timezone.utc</code>
Scope creep from group chat	P2 gated on explicit in-channel go-ahead only

6. Open questions for the reviewer

1. Is one-Episode-per-step the right granularity, or should the hook batch `N` steps into one cluster? (Current: fine-grained, cheap, audit-friendly; cost: journal grows ~ 1 cluster/step.)
2. Should the context transformation rank index entries (e.g. by About-tag relevance to the last user message) instead of pure recency-tail?
3. Compaction policy: when the journal exceeds `N` clusters, fold oldest Episodes into a summary Fact and archive the raw file — acceptable loss of raw granularity?
4. Any objection to Supersedes-only corrections (no deletion) as the permanent convention?